Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №4
по дисциплине «Инженерно-техническая поддержка сопровождения интеллектуальных информационных систем»
«Контейнеризация компонентов ИИС и управление зависимостями (Docker)»
ЛР 09.02.08.02.004

Студент группы 1ИИС-41 Н.М. Аникин
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Лабораторная работа №4

Контейнеризация компонентов ИИС и управление зависимостями (Docker)

Цели и задачи работы: освоение принципов контейнеризации интеллектуальных информационных систем (ИИС) с использованием Docker.

1. Установить Docker и Docker Compose (при отсутствии). Проверить версии команд docker --version и docker compose version.

2. Создать структуру каталога iis-docker и файлы, описанные в разделах 4–8.

3. Написать multi-stage Dockerfile на базе python:3.11-slim.

4. Создать файл .dockerignore.

5. Описать docker-compose.yml с сервисами api и db (PostgreSQL) и именованными volumes.

6. Выполнить docker compose build и зафиксировать размер итогового образа (docker images).

7. Запустить систему командой docker compose up -d.

8. Проверить работоспособность API с помощью curl или браузера.

9. Убедиться, что volume model_data смонтирован и модель сохраняется между перезапусками.

10. Выполнить docker compose down, затем снова docker compose up -d и подтвердить сохранность данных.

11. (Дополнительно) Добавить в docker-compose сервис Redis или Qdrant и использовать его для кэширования предсказаний.

Задание к работе: выполнить задания 1-11 в соответствии с условиями. Ответить на контрольные вопросы.

Задание принял:  Аникин Н.М.

Подпись

Ход работы

Ссылка на GitHub-репозиторий: https://github.com/itsqwit/anikin_1iis41

1. Установить Docker и Docker Compose (при отсутствии). Проверить версии команд docker --version и docker compose version.

```
anikin_1iis41@ubuntu26041:~$ sudo install -m 0755 -d /etc/apt/keyrings
anikin_1iis41@ubuntu26041:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
> -o /etc/apt/keyrings/docker.asc
anikin_1iis41@ubuntu26041:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
anikin_1iis41@ubuntu26041:~$
[0] 0:bash*
```

Рисунок 1 – добавление GPG-ключа Docker

```
anikin_1iis41@ubuntu26041:~$ echo "deb [arch=$(dpkg --print-architecture) \
> signed-by=/etc/apt/keyrings/docker.asc] \
> https://download.docker.com/linux/ubuntu \
> $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
> sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
anikin_1iis41@ubuntu26041:~$
```

Рисунок 2 – подключение репозитория Docker

```
anikin_1iis41@ubuntu26041:~$ sudo apt-get update
Hit:1 http://ru.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:3 http://ru.archive.ubuntu.com/ubuntu resolute-updates InRelease
Get:4 https://download.docker.com/linux/ubuntu resolute InRelease [32.5 kB]
Hit:5 http://ru.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 https://download.docker.com/linux/ubuntu resolute/stable amd64 Packages [32.3 kB]
Fetched 64.7 kB in 0s (166 kB/s)
Reading package lists... Done
anikin_1iis41@ubuntu26041:~$ sudo apt-get install docker-ce docker-ce-cli containerd.io \
> docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
```

Рисунок 3 – установка пакетов Docker

```
anikin_1iis41@ubuntu26041:~$ docker --version
Docker version 29.9.2, build a82e528
anikin_1iis41@ubuntu26041:~$ docker compose version
Docker Compose version v5.6.1
```

Рисунок 4 – проверка версий

2. Создать структуру каталога iis-docker и файлы, описанные в разделах 4–8.

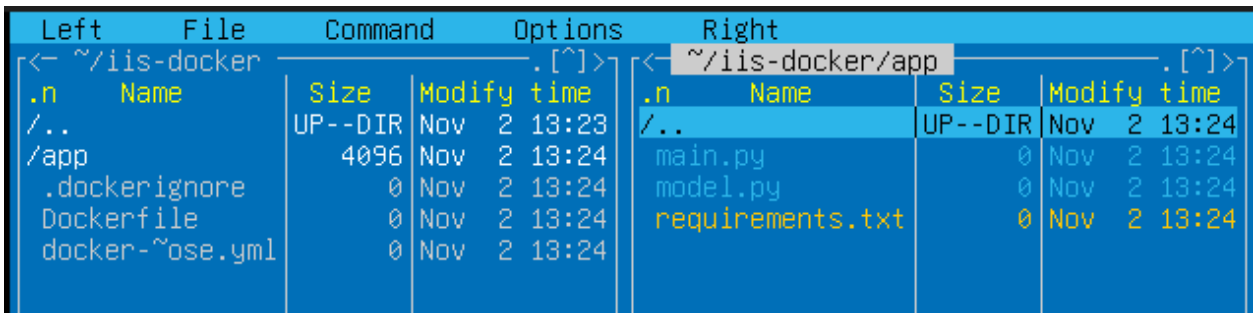


Рисунок 5 – создание структуры каталога и файлов



Рисунок 6 – редактирование requirements.txt

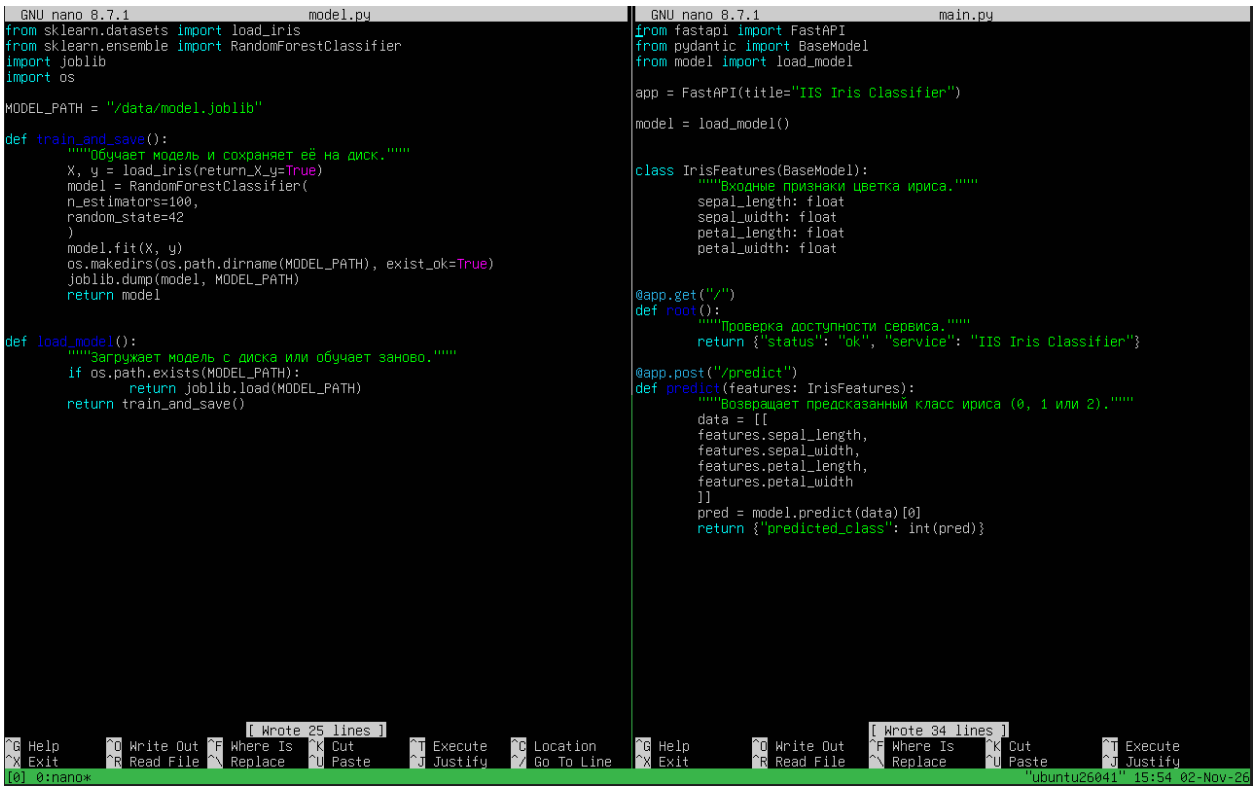


Рисунок 7 – редактирование main.py и model.py

3. Написать multi-stage Dockerfile на базе python:3.11-slim.

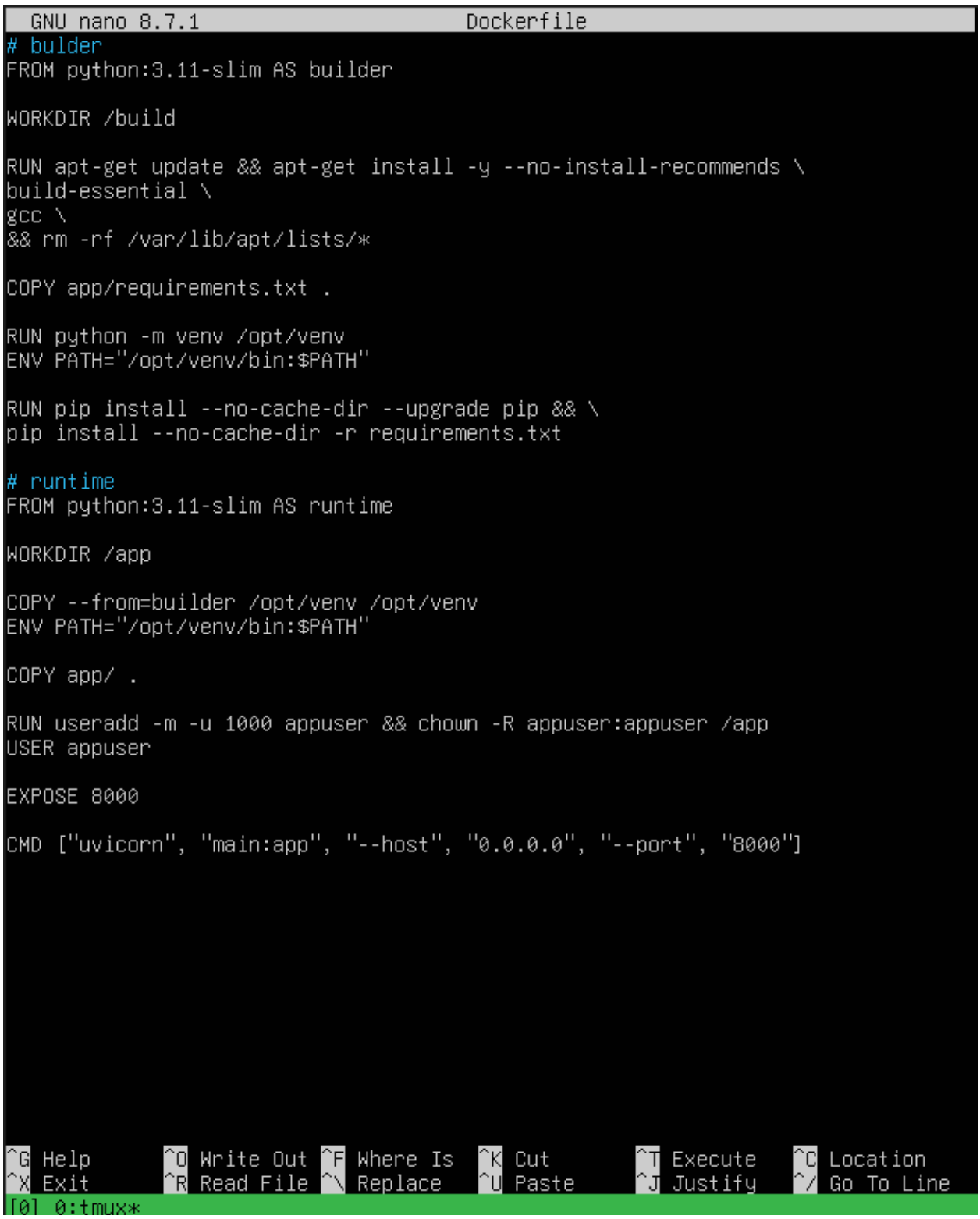


Рисунок 8 – редактирование Dockerfile

4. Создать файл .dockerignore.



Рисунок 9 – редактирование .dockerignore

5. Описать docker-compose.yml с сервисами api и db (PostgreSQL) и именованными volumes.

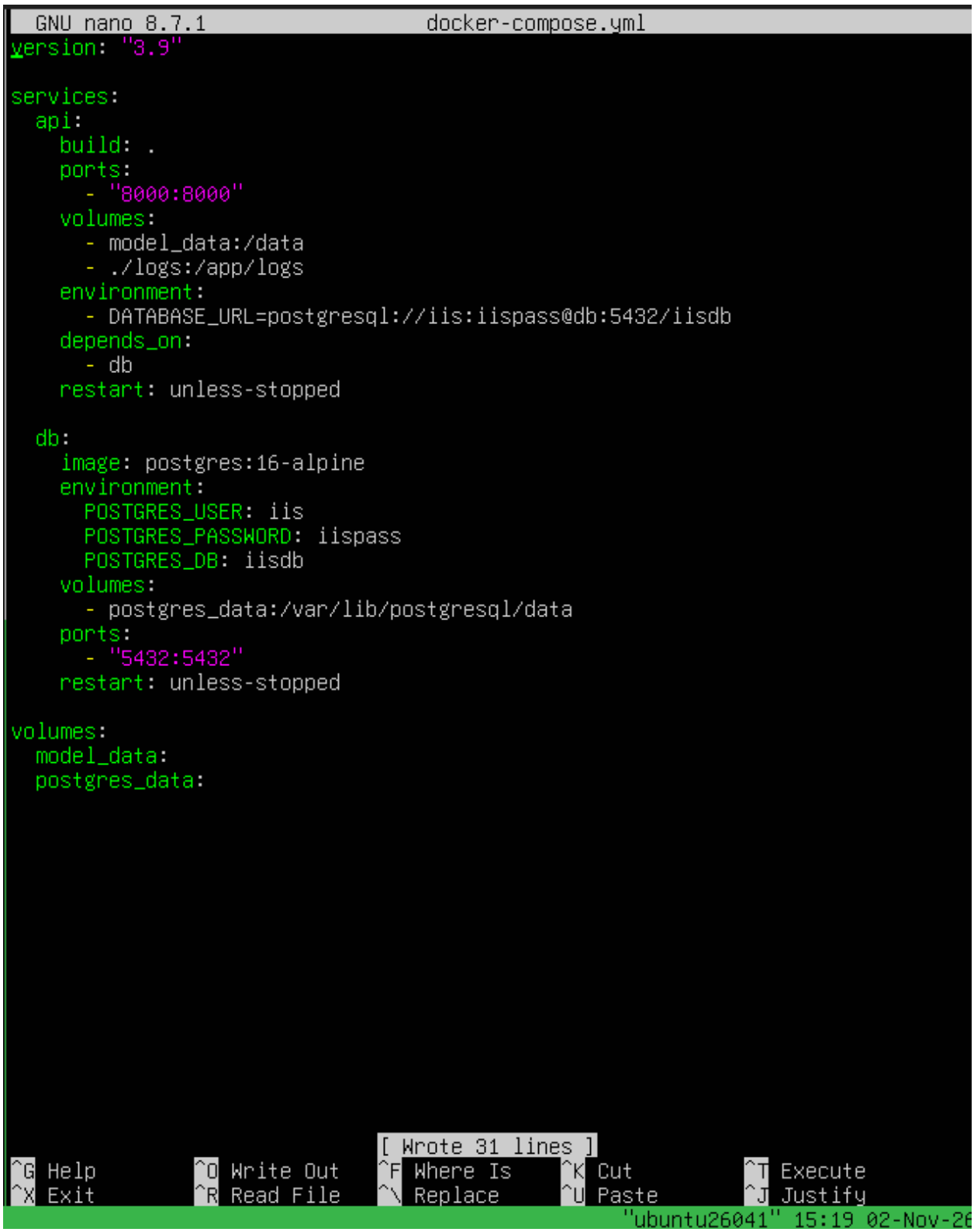


Рисунок 10 – редактирование docker-compose.yml

6. Выполнить docker compose build и зафиксировать размер итогового образа (docker images).

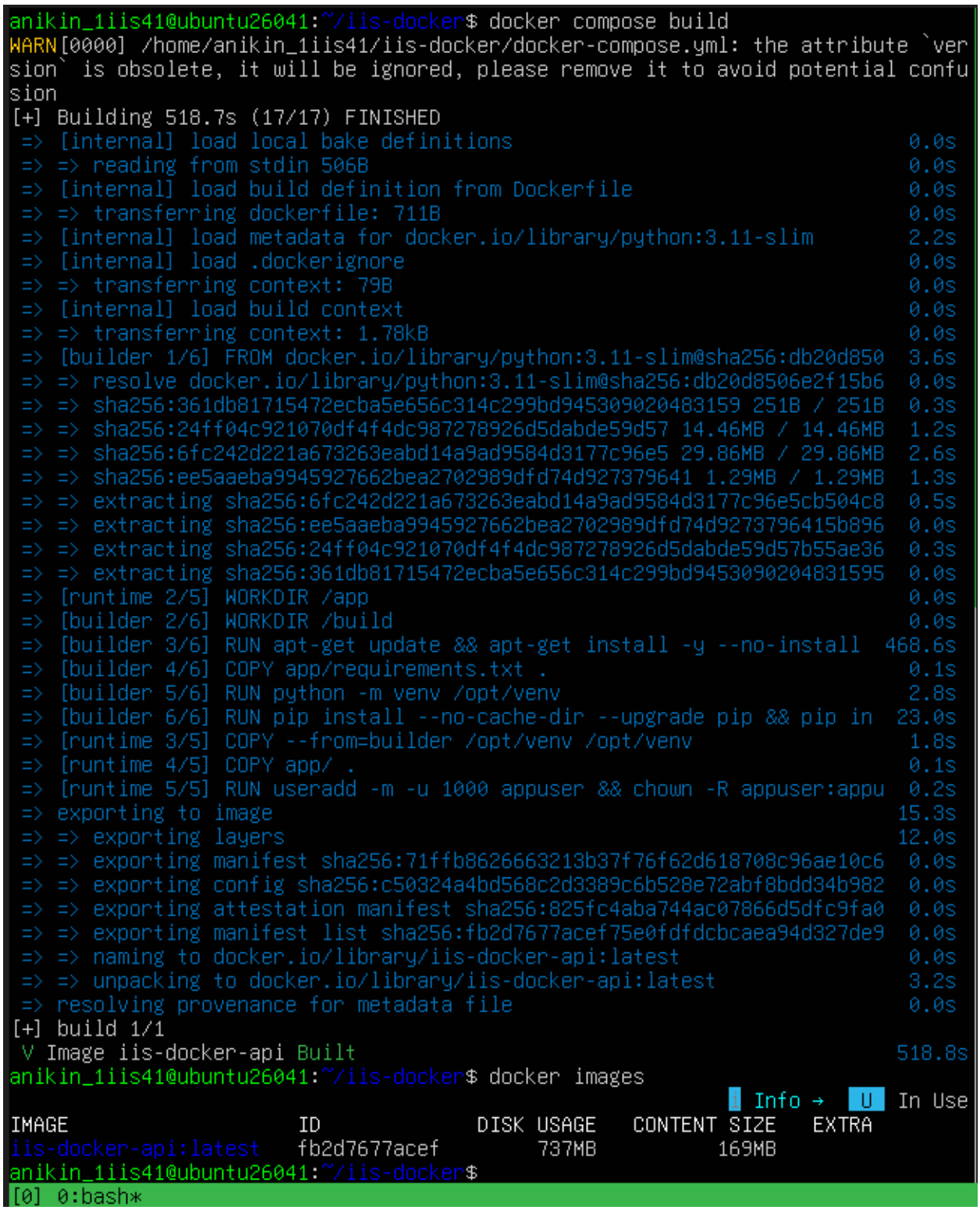


Рисунок 11 – сборка и размер образа

7. Запустить систему командой docker compose up -d

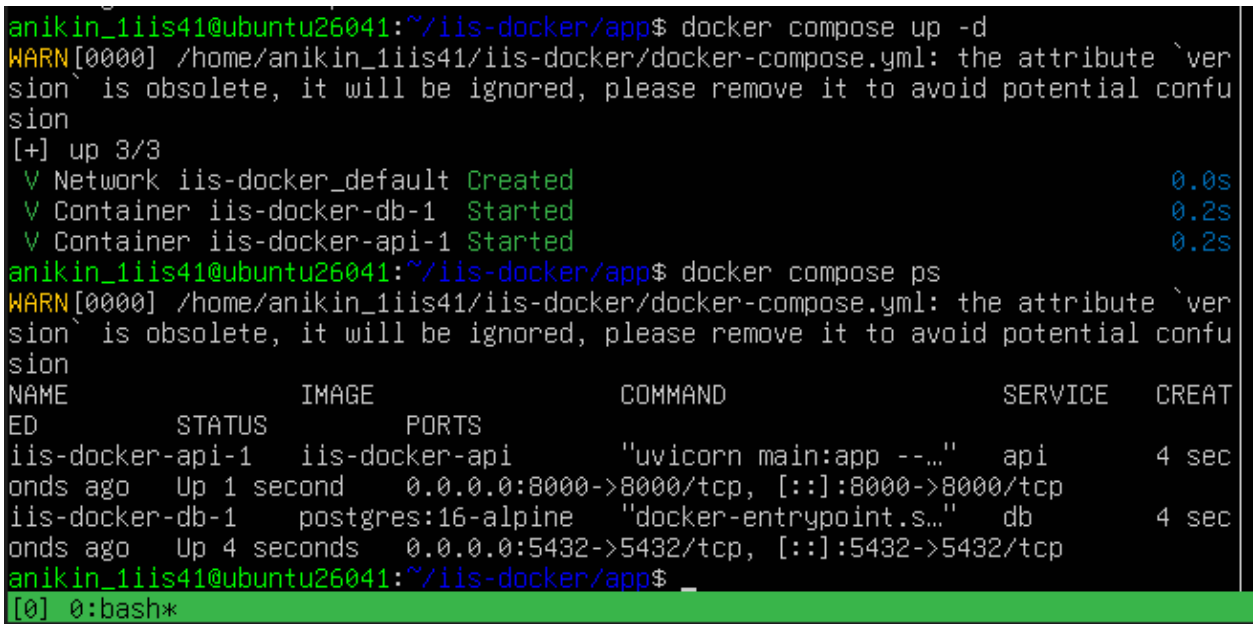


Рисунок 12 – запуск системы

8. Проверить работоспособность API с помощью curl или браузера.

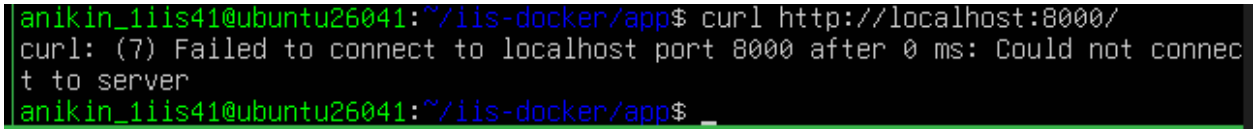


Рисунок 13 – проверка работоспособности с помощью curl: ошибка



Рисунок 14 – статус контейнера: api постоянно перезагружается



Рисунок 15 – лог ошибки

Ошибка происходит из-за того, что в Dockerfile я указал запуск приложения от имени непривилегированного пользователя appuser, однако для доступа к

папке /data нужны права root. Для решения этой проблемы я воспользовался командой docker compose run --rm --user root api chown -R 1000:1000 /data

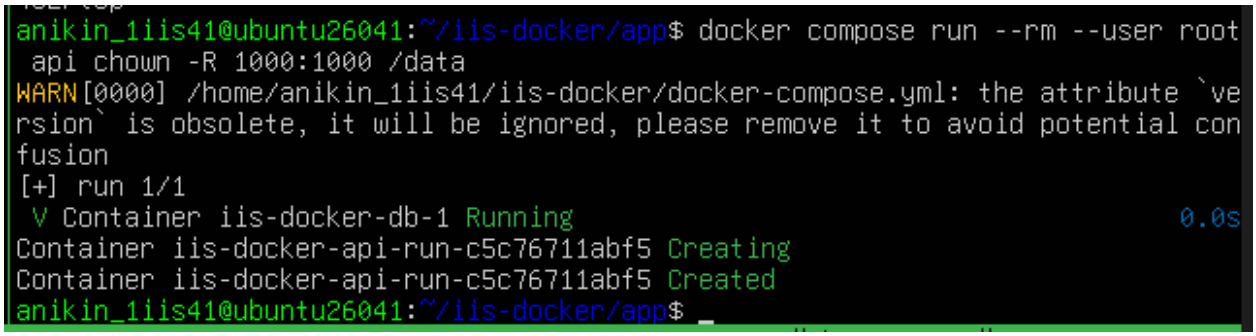


Рисунок 16 – исправление ошибки

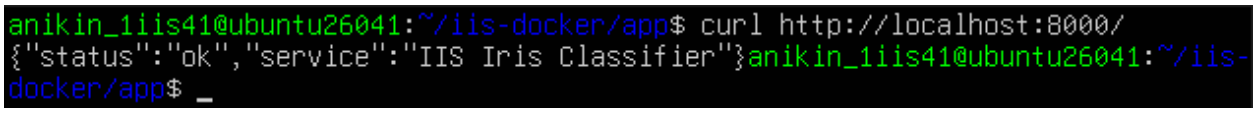


Рисунок 17 – проверка работоспособности с помощью curl: ошибка исправлена

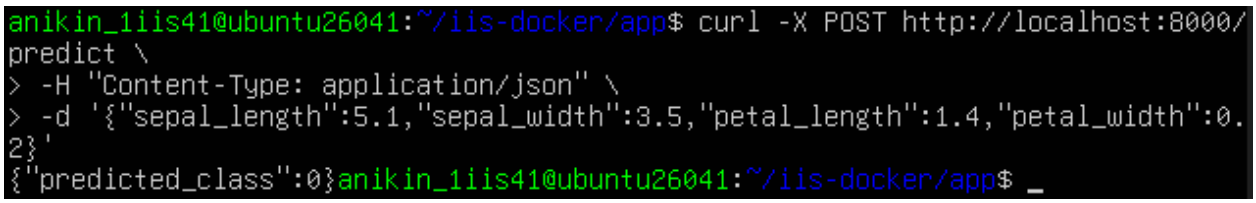


Рисунок 18 – запрос предсказания

9. Убедиться, что volume model_data смонтирован и модель сохраняется между перезапусками.

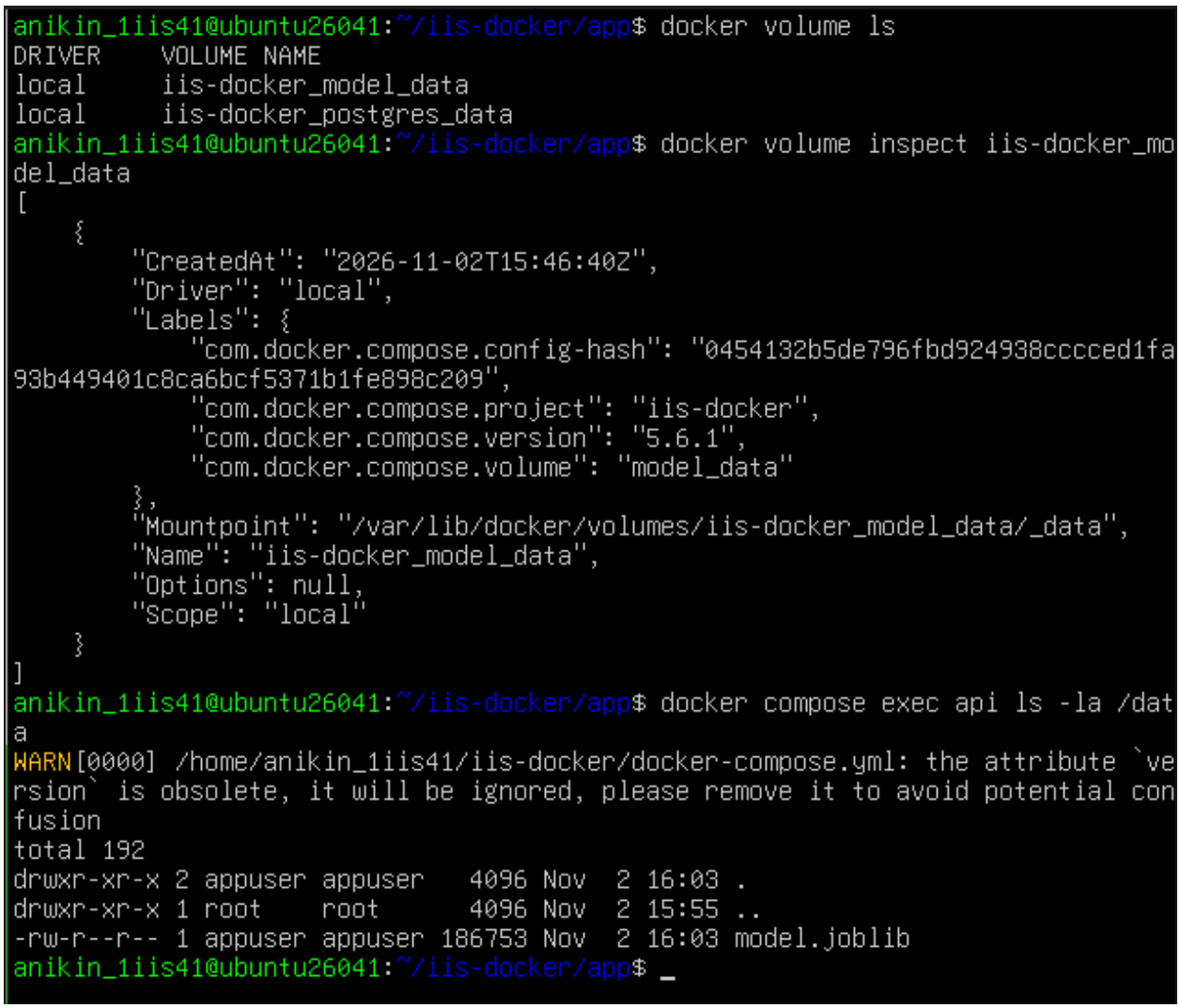


Рисунок 19 – информация о volume model_data

10. Выполнить docker compose down, затем снова docker compose up -d и подтвердить сохранность данных.

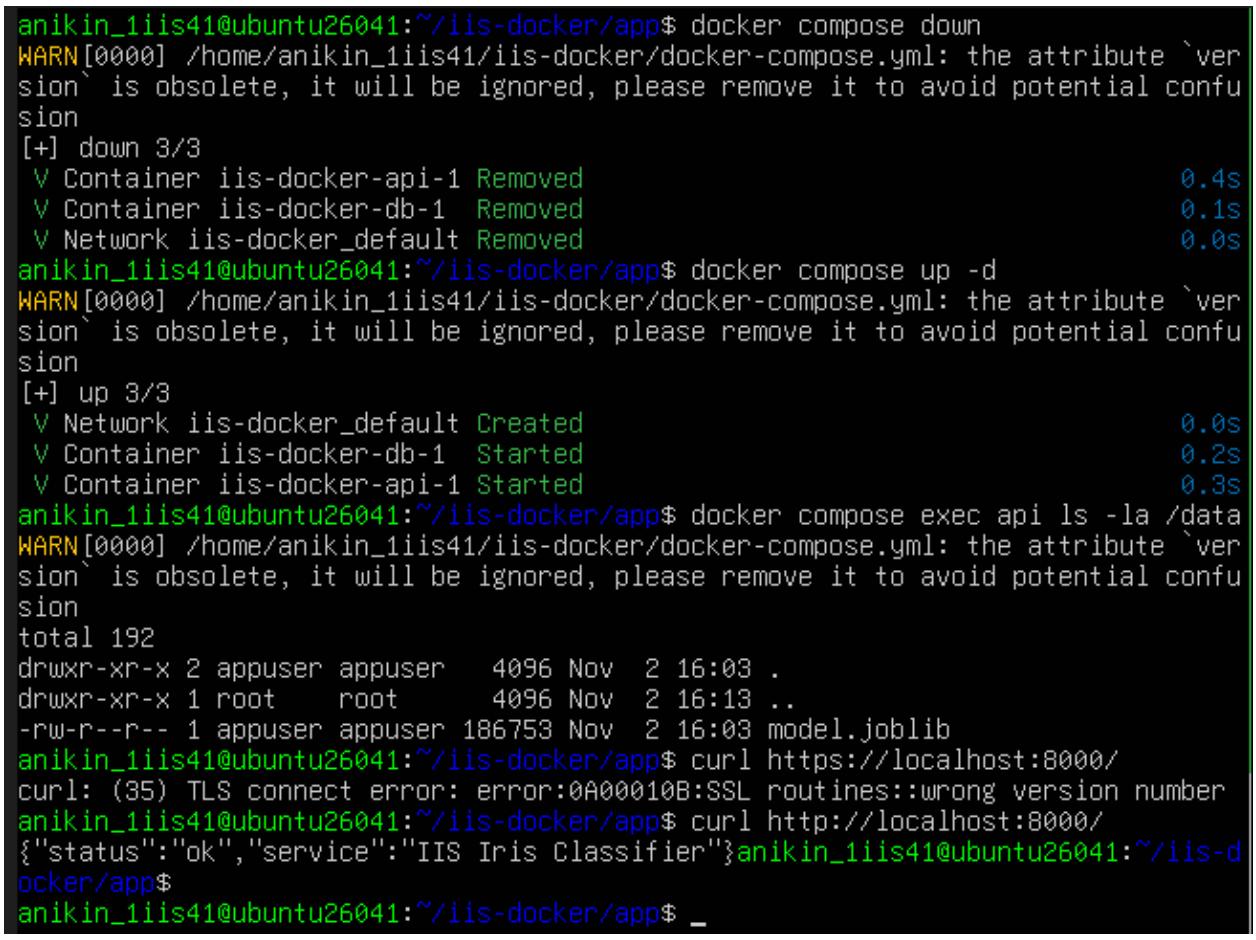


Рисунок 20 – перезапуск контейнера: файл model.joblib сохранился, работоспособность системы не нарушена

11. (Дополнительно) Добавить в docker-compose сервис Redis или Qdrant и использовать его для кэширования предсказаний.

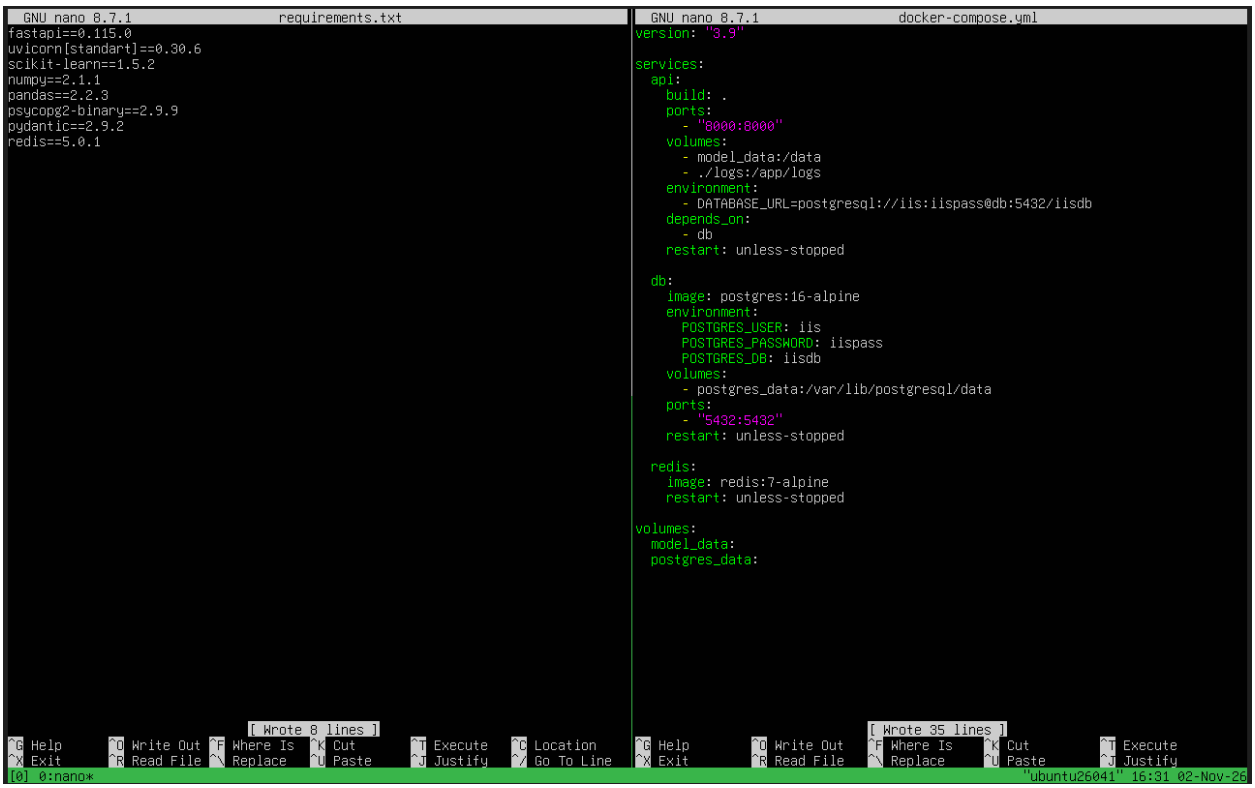


Рисунок 21 – редактирование requirements.txt и docker-compose.yml: добавлен сервис Redis

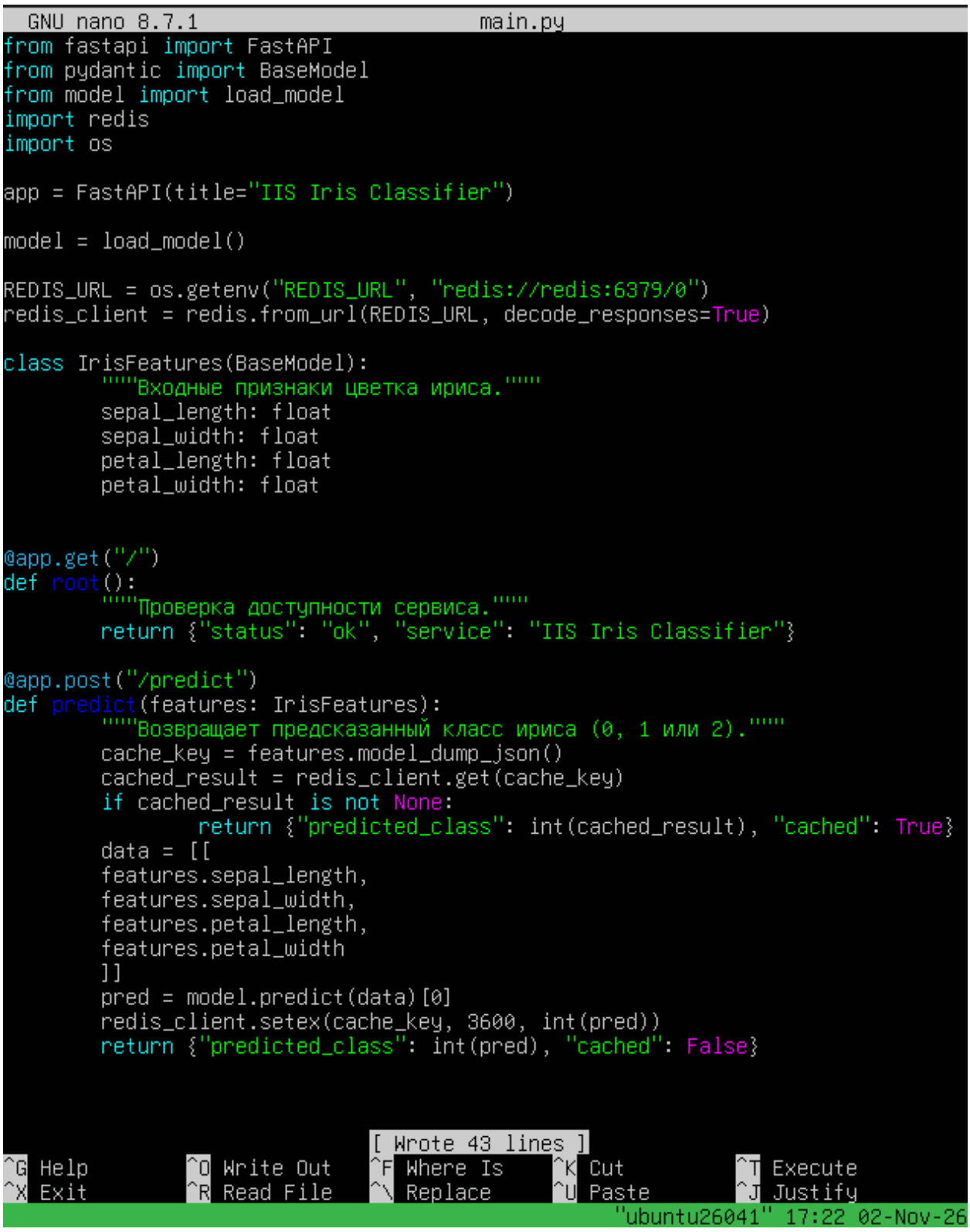


Рисунок 22 – редактирование main.py: добавлена логика кэширования предсказаний и поиска из уже кэшированных

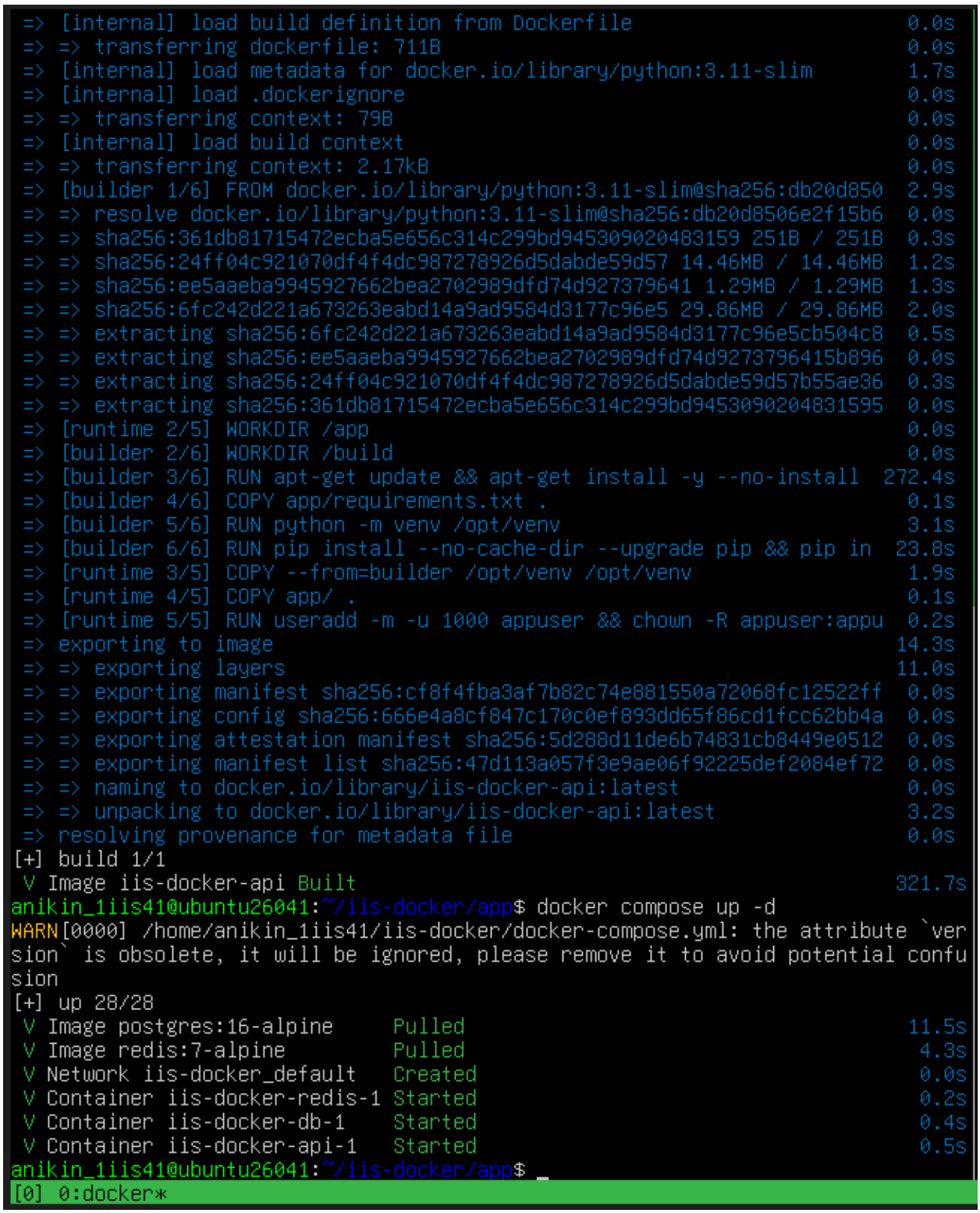


Рисунок 23 – пересборка образа и запуск контейнера

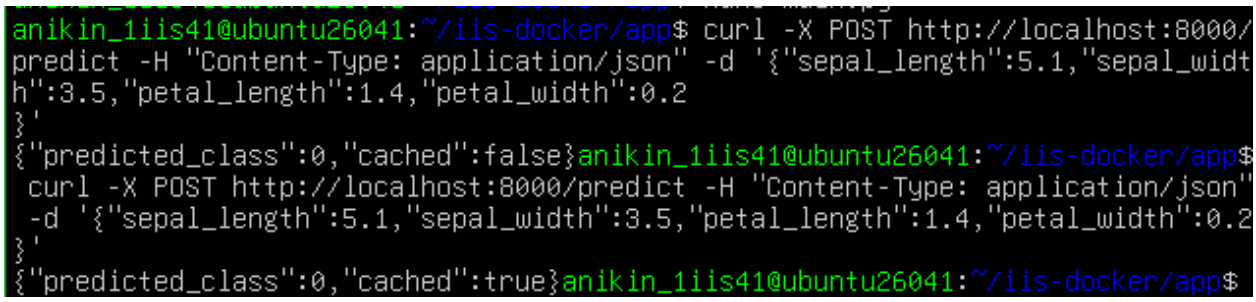


Рисунок 24 – запрос предсказания дважды

В первый раз модель рассчитывает ответ (“cached”:false), во второй раз API получает готовый ответ из оперативной памяти Redis по ключу (“cached”:true).

Контрольные вопросы

1. Что такое Docker-образ и чем он отличается от контейнера?

Образ — это неизменяемый шаблон, из которого создаются контейнеры. Контейнер — это запущенный экземпляр этого образа.

2. Для чего используется multi-stage build и какую проблему он решает при контейнеризации систем машинного обучения?

Multi-stage build — техника сборки, при которой в одном Dockerfile используется несколько стадий. На первой стадии устанавливаются зависимости и собирается приложение; на финальной стадии в лёгкий образ копируются только необходимые артефакты. Она решает проблему большого размера образов систем машинного обучения.

3. Почему рекомендуется использовать образы python:*-slim или alpine вместо полноразмерных?

Использование базовых образов slim или alpine вместо полноразмерных рекомендуется для уменьшения итогового размера Docker-образа.

4. Что такое слой (layer) в Docker и почему важен порядок инструкций в Dockerfile?

Слой (layer) — результат выполнения одной инструкции Dockerfile. Порядок инструкций влияет на эффективность кэширования при пересборке.

5. Для чего предназначен файл .dockerignore?

Файл .dockerignore предназначен для исключения лишних файлов из контекста сборки Docker.

6. В чём различие между bind-mount и named volume?

named volume — это именованный механизм персистентного хранения данных вне жизненного цикла контейнера, а bind-mount применяется для прямого связывания локальных каталогов хоста с контейнером.

7. Как обеспечить сохранение состояния модели и базы данных при пересоздании контейнеров?

Для сохранения состояния модели и базы данных при пересоздании контейнеров применяются volumes, которые монтируются к соответствующим каталогам внутри контейнеров.

8. Что выполняет инструкция COPY --from=builder?

Инструкция COPY --from=builder копирует конкретные собранные артефакты или каталоги из промежуточной стадии в текущую стадию.

9. Зачем запускать процесс приложения от имени непривилегированного пользователя?

Запуск процесса от имени непривилегированного пользователя необходим из соображений безопасности.

10. Как просмотреть логи отдельного сервиса в docker-compose?

Просмотреть логи отдельного сервиса в docker-compose можно с помощью команды docker compose logs <имя_сервиса>

11. Какими способами можно уменьшить размер образа при использовании PyTorch или TensorFlow?

Использование базовых образов slim/alpine, применение multi-stage build, использование флага --no-cache-dir при установке пакетов через pip, а также исключение лишних файлов через .dockerignore.

12. Что означает параметр depends_on и гарантирует ли он готовность зависимого сервиса?

Параметр depends_on определяет порядок запуска сервисов в многоконтейнером приложении, но он не гарантирует готовность зависимого сервиса к работе.

13. Как определить, какие volumes использует проект?

С помощью команды docker volume ls

14. В чём преимущество контейнеризации ИИС по сравнению с виртуальными машинами?

Контейнеризация позволяет упаковать ИИС со всеми конкретными версиями системных пакетов и тяжелых библиотек в изолированный контейнер, обеспечивая абсолютно одинаковую работу системы на машине разработчика, сервере и в облаке.

15. Какие риски возникают при хранении паролей и секретов непосредственно в docker-compose.yml?

Это создает риски утечки учетных данных при попадании файла в публичные репозитории.

Вывод

В ходе выполнения этой лабораторной работы я освоил принципы контейнеризации ИИС на примере веб-сервиса классификации ирисов. Я установил пакет docker и плагин docker-compose. Создал структуру каталога iis-docker и необходимые файлы, написал multi-stage Dockerfile. Это позволило получить итоговый образ размером 737 МБ. В файле docker-compose.yml я описал взаимодействие веб-сервиса на FastAPI и базы данных PostgreSQL. Для постоянного хранения модели я настроил именнованные хранилища model_data и postgres_data. При первом запуске контейнера я столкнулся с ошибкой наршения прав доступа, которую решил временным запуском контейнера под суперпользователем и сменой владельца директории. В рамках дополнительного задания я интегрировал сервис Redis в общую архитектуру образа. В код добавил логику кэширования. Благодаря этому результаты для повторных запросов стали извлекаться напрямую из оперативной памяти без повторных вызовов модели.